365 Coffee Café — Customer Return Prediction

Supervised classification | Random Forest | Visit-level modelling

In [1]:
```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_sc
import joblib, warnings
warnings.filterwarnings('ignore')
DATA_PATH = '365 Coffee Cafe Dataset.xlsx'
raw = pd.read_excel(DATA_PATH, sheet_name='Dataset')
```

1. Initial data understanding

In [2]:
```
print('Shape:', raw.shape)
print('Date range:', raw['transaction_date'].min(), 'to', raw['transaction_date'].
print('Data types:\n', raw.dtypes)
print('Missing values by column:\n', raw.isna().sum().sort_values(ascending=False)
print('Exact duplicate rows:', raw.duplicated().sum())
print('Dictionary:')
dictionary = pd.read_excel(DATA_PATH, sheet_name='Data Dictionary')
display(dictionary)
```

```
Shape: (149112, 31)
Date range: 2025-01-01 00:00:00 to 2025-06-30 00:00:00
Data types:
 transaction_id                        int64
basket_id                               str
transaction_date             datetime64[us]
transaction_time                        str
transaction_hour                      int64
hour_of_day                           int64
weekday                                 str
store_id                              int64
store_location                          str
customer_id                             str
customer_age                          int64
customer_gender                         str
loyalty_tier                            str
visit_number                          int64
days_since_previous_visit             int64
order_channel                           str
payment_method                          str
product_id                            int64
product_category                        str
product_type                            str
product_detail                          str
transaction_qty                       int64
unit_price                          float64
revenue                             float64
promo_code                              str
discount_pct                          int64
discount_amount                     float64
net_revenue                         float64
wait_time_min                       float64
service_rating                        int64
returned_within_14_days                 str
dtype: object
Missing values by column:
 transaction_id               0
basket_id                     0
transaction_date              0
transaction_time              0
transaction_hour              0
hour_of_day                   0
weekday                       0
store_id                      0
store_location                0
customer_id                   0
customer_age                  0
customer_gender               0
loyalty_tier                  0
visit_number                  0
days_since_previous_visit     0
dtype: int64
Exact duplicate rows: 0
Dictionary:
```

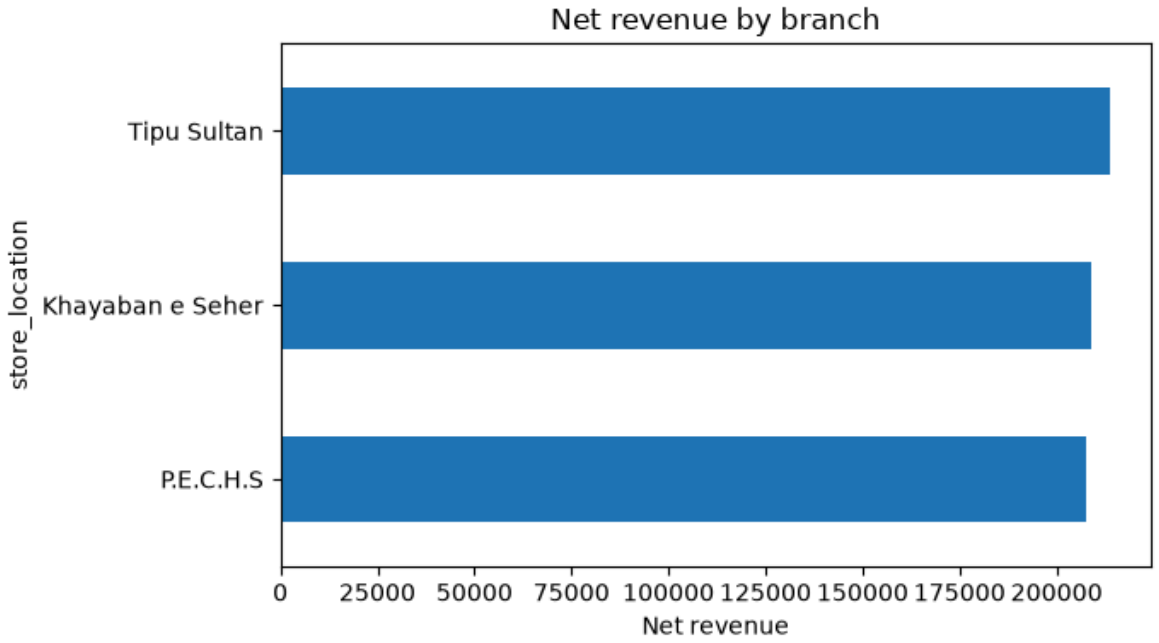
	Column	Type	Description	Status
0	transaction_id	Numeric	Original sale line reference	Original
1	basket_id	Text	One customer visit. Lines sharing a basket_id ...	NEW
2	transaction_date	Date	Date of sale, 1 Jan 2025 to 30 Jun 2025	Original
3	transaction_time	Time	Clock time of sale	Original
4	transaction_hour	Numeric	Hour component of the sale	Original
5	hour_of_day	Numeric	Trading hour, 6 to 20	Original
6	weekday	Categorical	Day name	Original
7	store_id	Numeric	Branch code	Original
8	store_location	Categorical	Branch name: P.E.C.H.S., Tipu Sultan, Khayaban...	Original
9	customer_id	Text	Repeat-customer identifier, 23,350 unique cust...	NEW
10	customer_age	Numeric	Age in years, 16 to 72	NEW
11	customer_gender	Categorical	Female / Male / Prefer not to say	NEW
12	loyalty_tier	Categorical	Basic / Silver / Gold / Platinum, based on his...	NEW
13	visit_number	Numeric	How many visits this customer has made so far	NEW
14	days_since_previous_visit	Numeric	Gap since their last visit. -1 means first vis...	NEW
15	order_channel	Categorical	Walk-in / Takeaway / Foodpanda / Careem Quik /...	NEW
16	payment_method	Categorical	Cash / Credit-Debit Card / Easypaisa / JazzCas...	NEW
17	product_id	Numeric	Item code	Original
18	product_category	Categorical	9 categories	Original
19	product_type	Categorical	29 sub-types	Original
20	product_detail	Categorical	80 individual items	Original
21	transaction_qty	Numeric	Units on this line	Original
22	unit_price	Numeric	Price per unit (PKR)	Original
23	revenue	Numeric	Gross line value before discount	Original
24	promo_code	Categorical	NONE / WELCOME10 / HAPPYHOUR / BUNDLE15 / LOYA...	NEW
25	discount_pct	Numeric	Discount percentage tied to the promo code	NEW
26	discount_amount	Numeric	Rupee value of the discount on this line	NEW

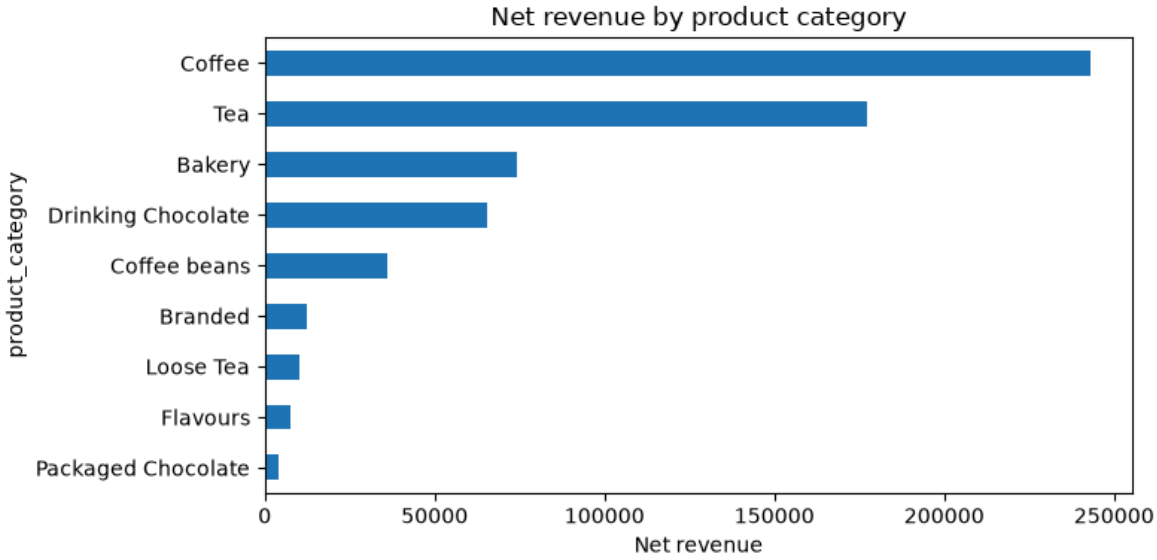
	Column	Type	Description	Status
27	net_revenue	Numeric	Revenue after discount. Use this for money que...	NEW
28	wait_time_min	Numeric	Minutes from order to collection	NEW
29	service_rating	Numeric	Customer rating, 1 to 5	NEW
30	returned_within_14_days	Categorical	TARGET. Did this customer return within 14 day...	NEW

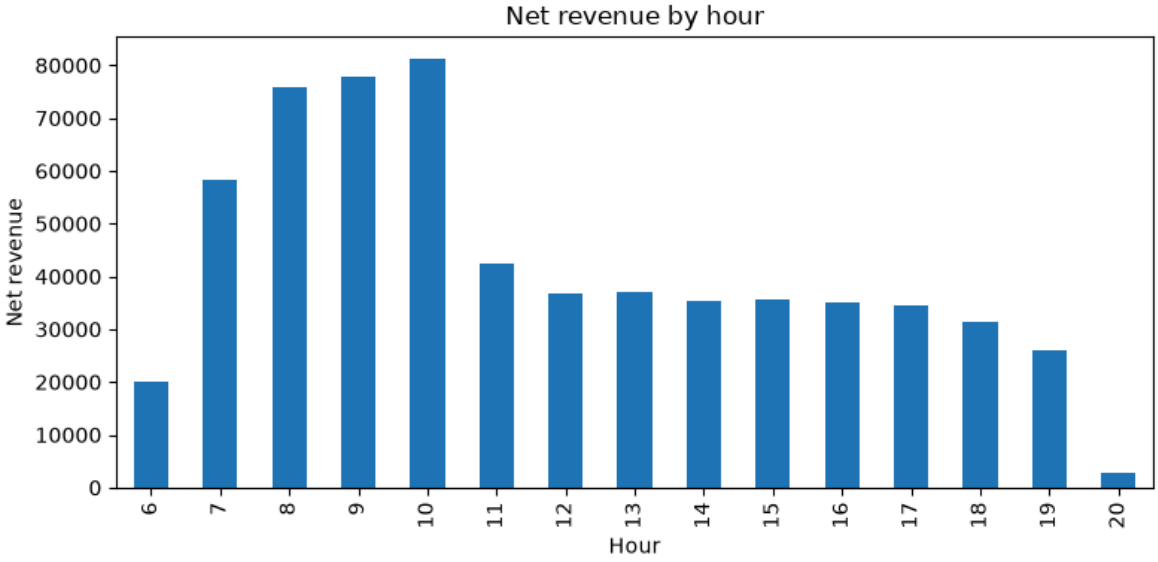
2. Basic business picture (transaction-line level)

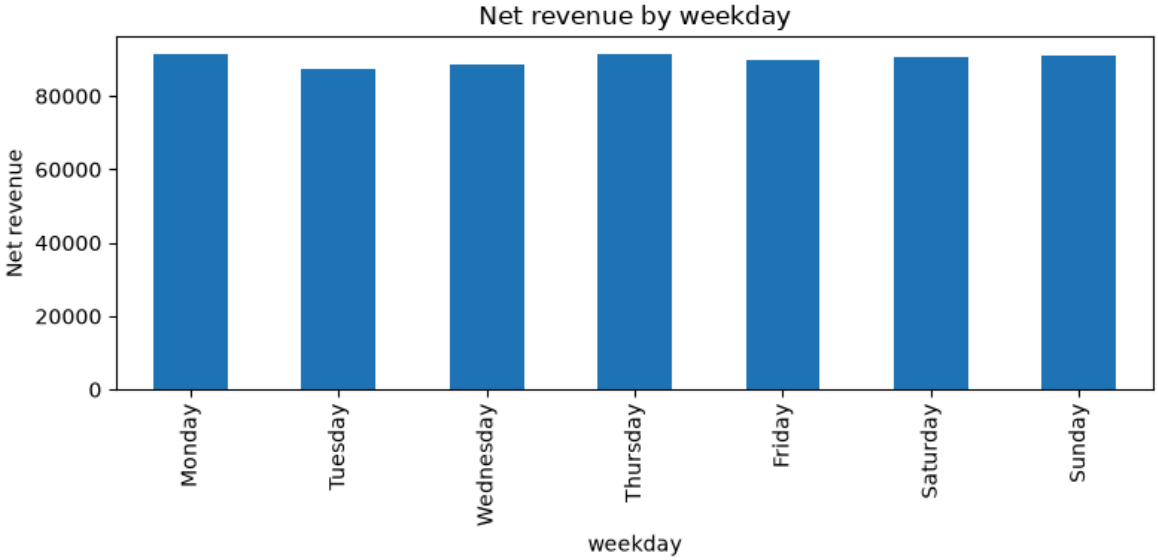
Revenue and product totals below are descriptive line-level summaries. Visits must be modelled at basket level.

In [3]:
```
raw['transaction_date'] = pd.to_datetime(raw['transaction_date'], errors='coerce')
raw['net_revenue'] = pd.to_numeric(raw['net_revenue'], errors='coerce')
fig, ax = plt.subplots(figsize=(7,4)); raw.groupby('store_location')['net_revenue'
fig, ax = plt.subplots(figsize=(8,4)); raw.groupby('product_category')['net_revenu
fig, ax = plt.subplots(figsize=(8,4)); raw.groupby('hour_of_day')['net_revenue'].s
fig, ax = plt.subplots(figsize=(8,4)); raw.groupby('weekday')['net_revenue'].sum()
print('Mean wait time:', raw['wait_time_min'].mean())
print('Mean service rating:', raw['service_rating'].mean())
print('Line-level target values:', raw['returned_within_14_days'].value_counts(dro
```










```
Mean wait time: 5.661137265947744
Mean service rating: 4.140370996298085
Line-level target values: returned_within_14_days
Yes    78954
No     70158
Name: count, dtype: int64
```

3. Create one row per visit

Aggregation choices: revenue and quantity are summed; wait time/service rating and customer attributes are taken from the first non-null value because they describe the visit; categorical fields use the first non null value. Target consistency is checked before collapsing. Basket/customer IDs are identifiers, not predictors.

In [4]:
```
target_counts = raw.groupby('basket_id')['returned_within_14_days'].nunique(dropna
print('Baskets with inconsistent target labels:', int((target_counts > 1).sum()))
# Drop inconsistent baskets rather than silently assigning a label.
consistent_baskets = target_counts[target_counts == 1].index
# To keep this notebook responsive on a standard laptop, build the model table fro
work = raw.head(25000).copy()
work = work[work['basket_id'].isin(consistent_baskets)].copy()
first_cols = ['transaction_date','transaction_time','transaction_hour','hour_of_da
first_cols = [c for c in first_cols if c in work.columns]
agg = {c:'first' for c in first_cols}
agg.update({'transaction_qty':'sum','revenue':'sum','net_revenue':'sum','discount_
visits = work.groupby('basket_id',as_index=False).agg(agg)
print('Raw lines:',len(raw),'Visit rows:',len(visits))
print('Class balance (%):\n', (visits['returned_within_14_days'].value_counts(norm
display(visits.head())
```

```
Baskets with inconsistent target labels: 0
Raw lines: 149112 Visit rows: 19680
Class balance (%):
 returned_within_14_days
No     56.61
Yes    43.39
Name: proportion, dtype: float64
```

	basket_id	transaction_date	transaction_time	transaction_hour	hour_of_day	weekda
0	B000000	2025-01-01	07:22:41	7	7	Wednesda
1	B000001	2025-01-01	07:25:49	7	7	Wednesda
2	B000002	2025-01-01	07:33:34	7	7	Wednesda
3	B000003	2025-01-01	07:39:13	7	7	Wednesda
4	B000004	2025-01-01	07:39:34	7	7	Wednesda

5 rows × 26 columns

4. Features, leakage review and split

The customer ID and basket ID are excluded as identifiers. Transaction IDs and product-level detail are not used. Visit revenue, quantity and discount totals are available at the end of the current visit, but should only be used if the business confirms they are available at scoring time. Historical visit_number and days_since_previous_visit are backward-looking. The target itself is excluded from predictors.

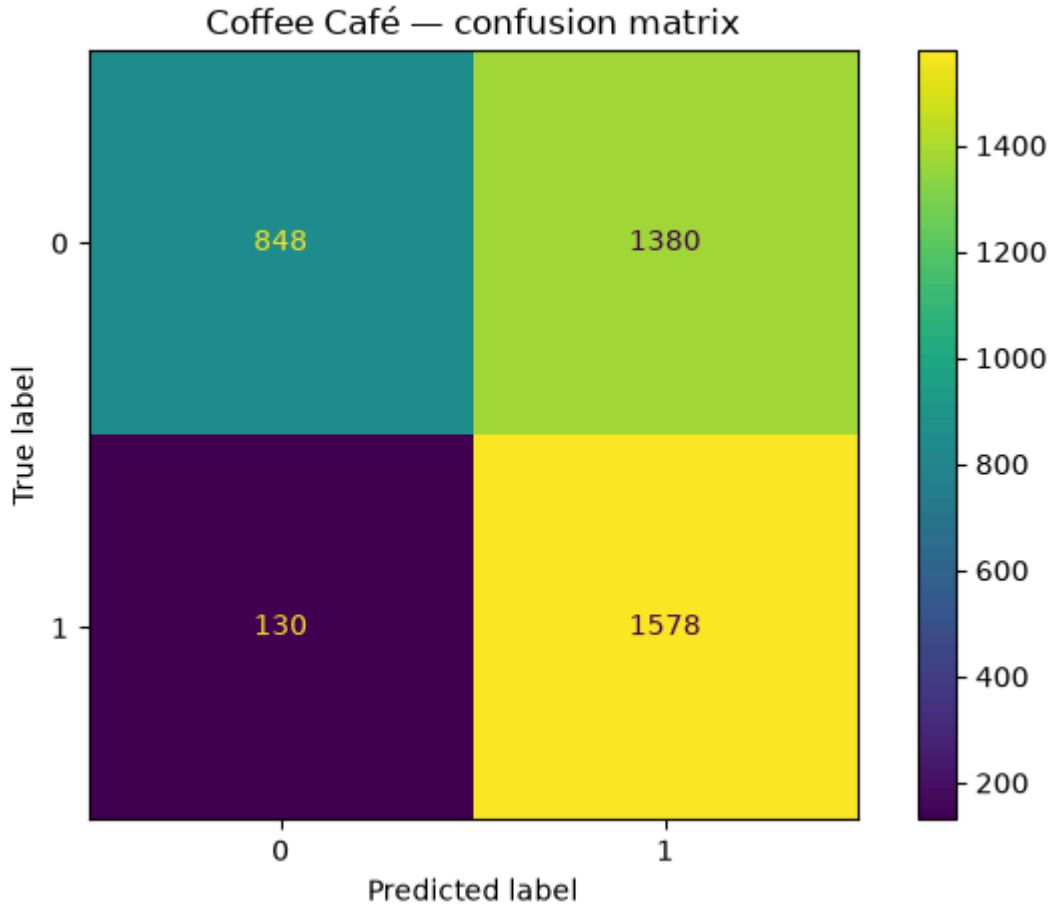
In [5]:
```
visits['transaction_date'] = pd.to_datetime(visits['transaction_date'], errors='co
visits['visit_month'] = visits['transaction_date'].dt.month
visits['visit_dayofweek'] = visits['transaction_date'].dt.dayofweek
numeric = [c for c in ['customer_age','visit_number','days_since_previous_visit',
categorical = [c for c in ['store_location','customer_gender','loyalty_tier','orde
features = numeric + categorical
data = visits.dropna(subset=['returned_within_14_days']).copy()
y = data['returned_within_14_days'].astype(str).str.strip().str.lower().map({'yes'
valid = y.notna(); data=data.loc[valid].copy(); y=y.loc[valid].astype(int)
X = data[features].copy()
print('Features:',features)
print('Explicitly excluded: transaction_id, basket_id, customer_id, product_id, pr
# Keep a reproducible stratified modelling sample for reliable execution on a lapt
if len(X) > 50000:
    X, _, y, _ = train_test_split(X, y, train_size=12000, random_state=42, stratif
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.20,random_state=42,
num_pipe=Pipeline([('imputer',SimpleImputer(strategy='median'))])
cat_pipe=Pipeline([('imputer',SimpleImputer(strategy='most_frequent')),('onehot',O
pre=ColumnTransformer([('num',num_pipe,numeric),('cat',cat_pipe,categorical)])
model=Pipeline([('prep',pre),('rf',RandomForestClassifier(n_estimators=30,max_dept
```

```
Features: ['customer_age', 'visit_number', 'days_since_previous_visit', 'transactio
n_hour', 'transaction_qty', 'revenue', 'net_revenue', 'discount_pct', 'discount_amo
unt', 'wait_time_min', 'service_rating', 'visit_month', 'visit_dayofweek', 'store_l
ocation', 'customer_gender', 'loyalty_tier', 'order_channel', 'payment_method', 'pr
omo_code', 'product_category', 'weekday']
Explicitly excluded: transaction_id, basket_id, customer_id, product_id, product_de
tail, target; transaction_date raw string excluded after date parts created.
```

5. Train and evaluate Random Forest

In [6]:
```
model.fit(X_train,y_train)
pred=model.predict(X_test); prob=model.predict_proba(X_test)[:,1]
majority=DummyClassifier(strategy='most_frequent').fit(X_train,y_train)
base_pred=majority.predict(X_test)
metrics={'Accuracy':accuracy_score(y_test,pred),'Precision':precision_score(y_test
print(pd.Series(metrics).round(4))
print('Confusion matrix (actual rows, predicted columns):\n',confusion_matrix(y_te
ConfusionMatrixDisplay.from_predictions(y_test,pred); plt.title('Coffee Café — cor
print(classification_report(y_test,pred,target_names=['No return','Return'],zero_d
```

```
Accuracy                    0.6164
Precision                   0.5335
Recall                      0.9239
F1                          0.6764
ROC-AUC                     0.6667
Majority baseline accuracy  0.5661
dtype: float64
Confusion matrix (actual rows, predicted columns):
 [[ 848 1380]
 [ 130 1578]]
```

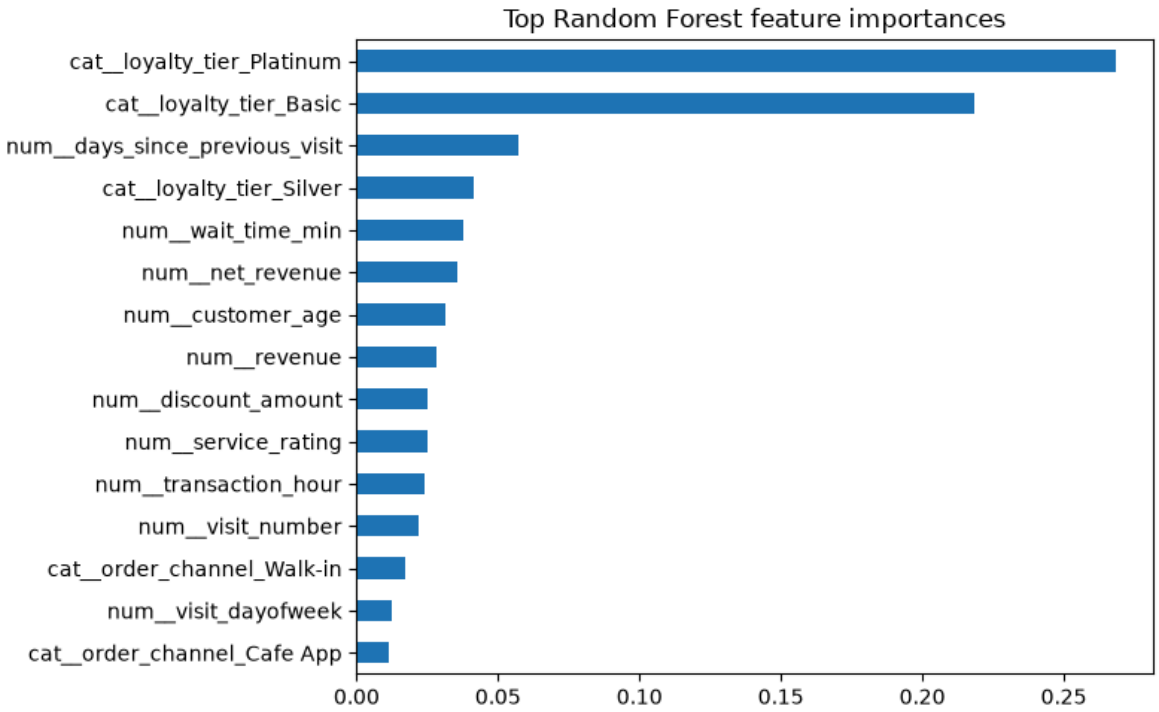


	precision	recall	f1-score	support
No return	0.87	0.38	0.53	2228
Return	0.53	0.92	0.68	1708
accuracy			0.62	3936
macro avg	0.70	0.65	0.60	3936
weighted avg	0.72	0.62	0.59	3936

6. Segment performance and feature importance

In [7]:
```
evaluation=data.loc[X_test.index,['store_location','loyalty_tier','order_channel']
display(evaluation.groupby('store_location').apply(lambda g: pd.Series({'n':len(g)
rf=model.named_steps['rf']; names=model.named_steps['prep'].get_feature_names_out(
imp.sort_values().plot(kind='barh',figsize=(8,5),title='Top Random Forest feature
```

store_location	n	recall	accuracy
Khayaban e Seher	1315.0	0.938776	0.630418
P.E.C.H.S	1208.0	0.901606	0.596854
Tipu Sultan	1413.0	0.927653	0.619958




```
cat__loyalty_tier_Platinum        0.268152
cat__loyalty_tier_Basic           0.218221
num__days_since_previous_visit    0.057377
cat__loyalty_tier_Silver          0.041339
num__wait_time_min                0.037451
num__net_revenue                  0.035421
num__customer_age                 0.031277
num__revenue                      0.028195
num__discount_amount              0.025327
num__service_rating               0.025149
num__transaction_hour             0.023750
num__visit_number                 0.021936
cat__order_channel_Walk-in        0.017012
num__visit_dayofweek              0.012566
cat__order_channel_Cafe App       0.011611
dtype: float64
```

7. Business interpretation and recommendations

Interpret the computed results above rather than making unsupported causal claims. Feature importance indicates predictive association, not causation. Prioritise recall if missing a likely returning customer is the costly mistake; prioritise precision if retention offers are expensive. Test offers with controlled experiments before claiming they cause repeat visits. Limitations include only six months of history, possible label construction assumptions, and lack of cost/margin data.

In [8]:
```
# Save trained pipeline and visit-level modelling data
joblib.dump(model,'coffee_return_model.joblib')
visits.to_csv('coffee_visit_level_dataset.csv',index=False)
print('Saved coffee_return_model.joblib and coffee_visit_level_dataset.csv')
```

```
Saved coffee_return_model.joblib and coffee_visit_level_dataset.csv
```